

Strategy Game Project Plan

Atte Inki, Oliver Niemi, Aleksi Sarja, Konsta Suuriniemi and Topi Varuskivi

General Project scope

The objective of the project is to create a turn-based Civilization-like strategy-game that utilizes Rogue-like deck builder elements. In broader terms this means building a game that is centered around a turn-based strategy where the player builds and manages an empire and its resources which are tied to handling a deck of playable cards, the moves of other players and different temporary effects on the player. These effects are based on cards, other players and the position in the game world.

The game should have most of the fundamental features of both rogue-like and civilization-type games and will progress in a turn-based format where the player can perform actions on each turn. The goal of the game is to defeat all other players by destroying their kingdoms.

Development is to be carried out with further expansions in mind. Consideration will be put into building a solid base for the game, with emphasis on good fundamental principles such as memory management and abstraction. The different gameplay-elements will be developed to be easily extensible.

Gameplay features

The game is played on a world constructed from hexagonal tiles. Only a limited number of tiles will be visible to the player at any time. The tiles will contain different terrain features based on the terrain type the tile has. Different terrain types contain different resources and alter, for example, the number of tiles visible and costs to move units.

The game will be non-hardcoded and should generate randomly according to a set of parameters. The exact random generation algorithm to be used will be decided later in development depending on deadline constraints. In a game, each player will start at a randomized location relatively far away from all other players. This way, the players will have time to develop their kingdom and explore their surroundings before engaging in combat with other players.

A few different types of buildings and units will also be implemented. The buildings will have features such as passive actions the building completes every turn, cost in resources and effects on other buildings. One of the buildings is a capital building, the destruction of which will cause the player to lose the game. Therefore, the goal of the game is to destroy the capital building of other players while defending your own.

Units are task performing agents, which can be placed on the map and commanded to perform some tasks such as moving on the map or attacking other units. Units will primarily be used for warfare. Both buildings and units will have health points, and if these health points are completely depleted the building or unit is destroyed. Units can deal damage to other units and buildings, and some units will be able to restore this damage.

During the game, the players will take turns. Each turn a player draws a new hand of playable cards that affect the gameplay in unique ways. The cards can be used to construct buildings, deploy units, and place effects upon players. Each player will have their own deck of cards which will be modified during the game. The initial plan is to simply give players an opportunity to modify their deck every third turn or so. Other possibilities for deck modification opportunities during the game will be considered once further in development.

Interface and usage

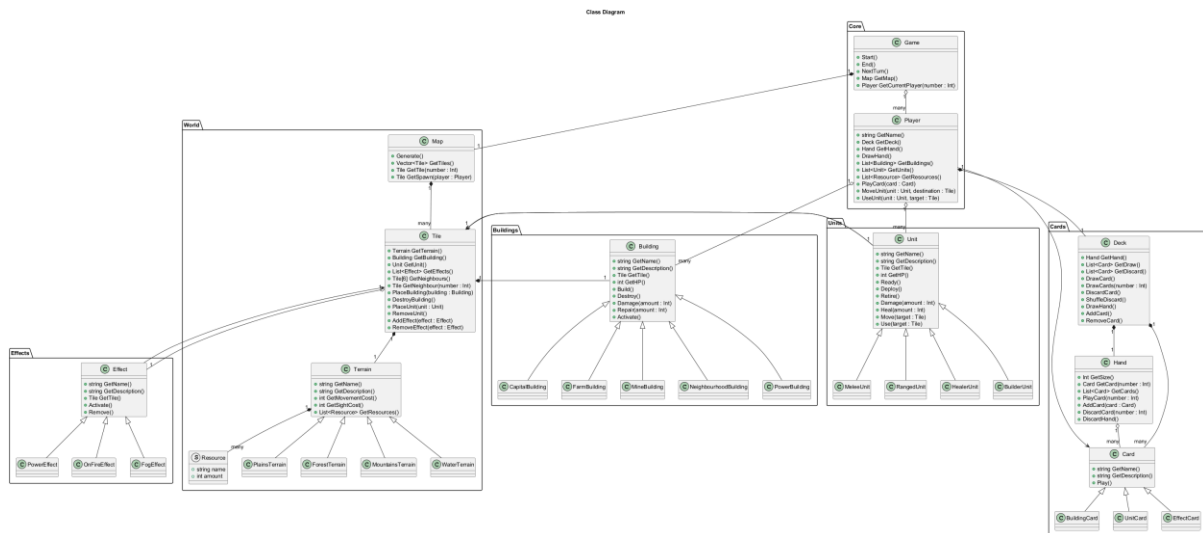
The player interacts with the game using their mouse through a visual interface. The game will include a simple main menu from which a game can be created or resumed. During a game, the player will be able to click on cards, buildings and units to manipulate them. Keyboard shortcuts may be considered. The world should be constructed from single color tiles while units and buildings will be represented by simple shapes. Additional features like simple sound effects and background music, better graphics and a visual map editor will be considered once these basic UI features have been put together.

A game state should be able to be saved to a file and loaded from it to save the player's progress between sessions. Multiplayer gameplay will be implemented locally on the same computing system. In multiplayer-mode two players can compete against each other turn-by-turn. AI-controlled single-player opponents are considered with online multiplayer being a stretch goal once these features have been implemented.

High level structure of the project

The initial project plan includes the class diagram for the most important backend classes of the game. On top of these classes there is the UI, but this is omitted in the class diagram as we are still learning how the SFML-library is used.

Additionally, the initial class diagram only displays the public functions of classes since it primarily aims to show the relationships and interactions between different classes. Exact private variables and functions will be decided later in development. The following diagram is the initial plan for the project class structure:



The planned use of external libraries

We expect to only use the SFML library for creating the user interface of the game. No other external libraries are planned to be used in the project.

Detailed sprint plans

The features marked with **green** are the ones we aim to implement. If we have enough time our aim is to choose some of the **yellow** ones for realization as well. The amount of work required for the implementation of each feature is a rough estimate and these estimates will be re-evaluated through project development.

Feature	Sprint	Amount of work (1–5)
Map class basics	1	1–2
Game state saving & loading (make easily scalable)	1–2	2–4
Game class basics	1–2	1–3
UI basics	1–2	3–5
Define tile types	1–2	2
Interactive UI (map movement)	2	3–4
Deck class and cards	2–3	2–3
Structure types & textures	2–3	2–3
Unit class	2–3	2
Player class	2–3	2–3
Local multiplayer	2–3	1–3
Map random generation	3	1–2
Interactive UI (playing cards)	3	2–4
Online multiplayer	3	1–4
Partially visible world map	3	1–2
AI controlled player	3	1–3
Map editor	3	2–4
Simple sfx & music	3	1–2
Better graphics	3	2–4

Project practices

This project uses GitLab as the primary version control platform. GitLab also offers an issue board which will be used to organize the workload and enhance project transparency. Our aim is to use a different issue board for each sprint to avoid unnecessary confusion between sprints. The features to be implemented in the sprint will be added to the issue board of the respective sprint during the sprint planning.

The meetings throughout the project will be scheduled well in advance, ideally already during the previous meeting. The team will use an internal telegram chat to discuss corrections to the meeting schedules. The same chat will be used to inform the team members about contributions made to the project as well as research results and notes. The meeting notes will be documented primarily by Oliver.

Our aim is to follow the standard C++ working practices. These include having a proper project structure, writing easily understandable code to ease debugging, not pushing faulty code to the remote repository, using common naming standards, checking for memory leaks and using header guards in header files. The implementation process starts with writing the most important header files and after that implementing the more detailed functionalities further. The connection between the branches and the items on the issue board should be easily understandable to increase the transparency of the project.

The project will be documented using Doxygen and clarifying inline comments. The testing will focus on the most crucial parts of the program, and it will be carried out using unit tests. The rest of the testing and debugging can be done using the UI and therefore our aim is to have the UI working as early as possible.